

Abdul Gafoor Mubashir
Assignment 2

Abstract

Learning to encrypt files in Asymmetric and Symmetric encryption. Sharing ascii format encrypted files and looking for captured data packets while sharing. Creating public and private keys and signing files with our private key and making the file encrypted with the public key of another person. Doing Steganography on image files and comparing their hash files to see if any modification has been made. The results demonstrate the effectiveness of these tools for securing and verifying data providing both integrity and authenticity.

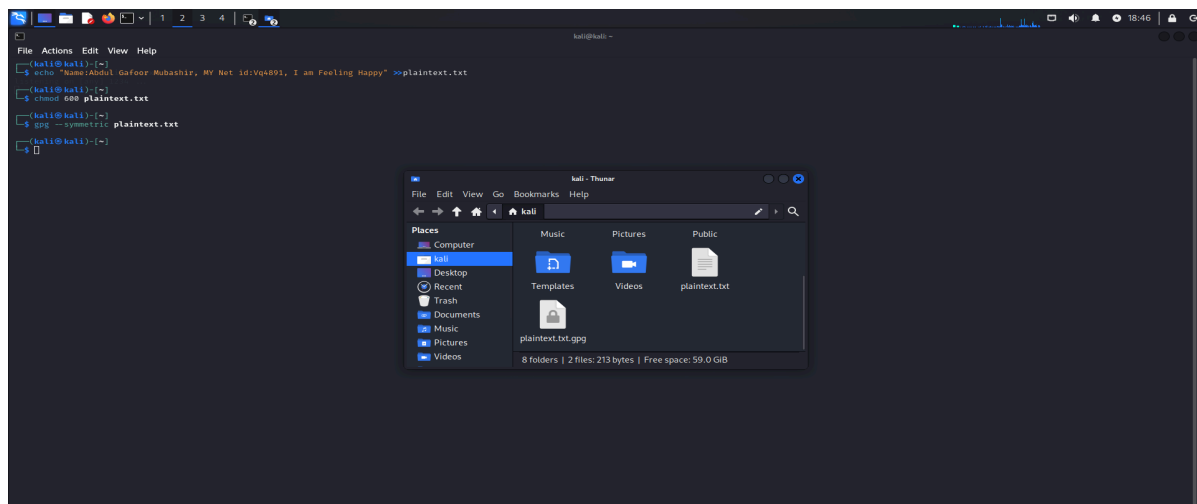
Introduction

Using GPG (GNU Privacy Guard) for symmetric and asymmetric encryption. Using Netcat for data transmission on the local network & Wireshark for packet analysis; and using Steghide for steganography & md5sum for integrity verification.

All used Commands are written and attached in the commands.txt file.

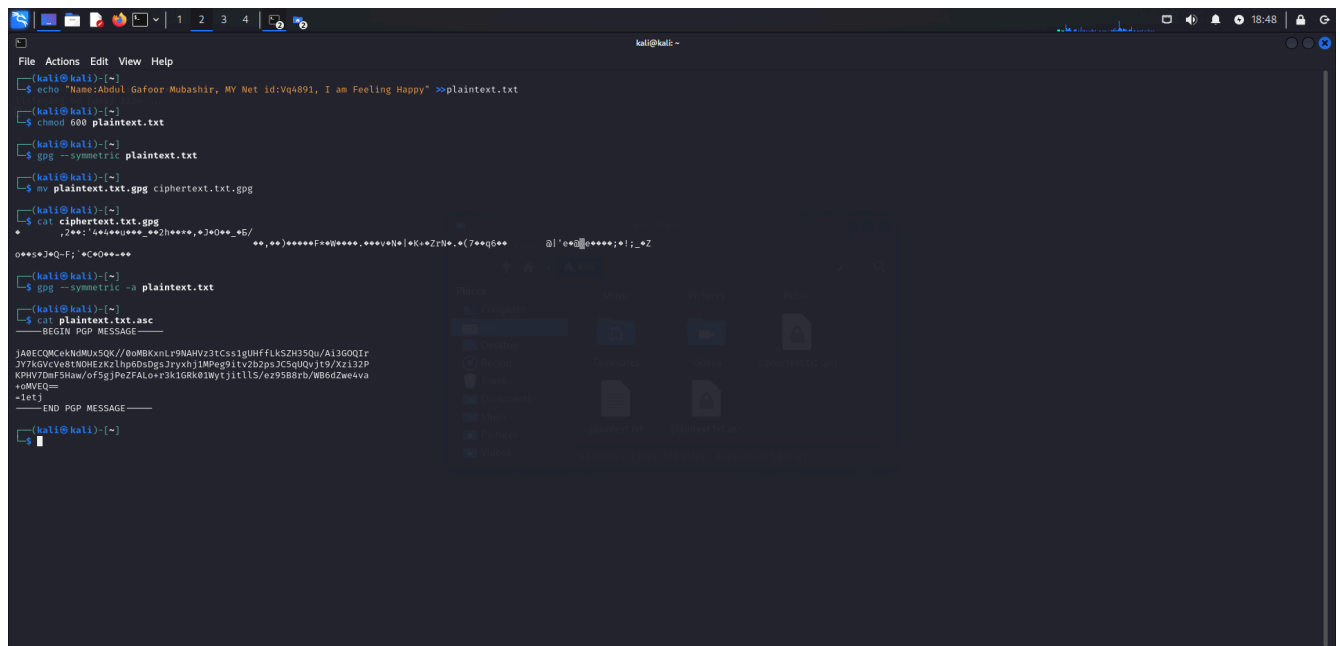
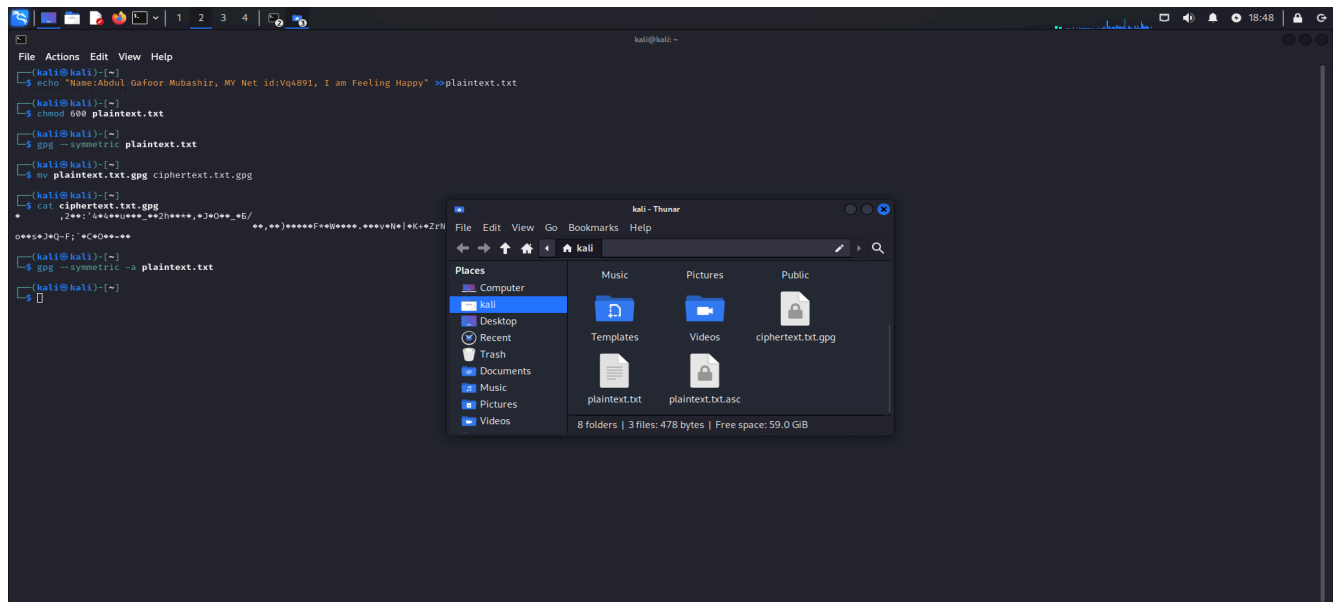
Summary of Results:

Opening bash terminal in kali linux and creating a file using echo command with our Name, Netid & a message and saving it as plaintext.txt. Changing file permission to 600 which makes it read and writable only by the file's owner. Now using GPG(GNU Privacy Guard) to do symmetric encryption to the plaintext.txt file. This file will be encrypted in binary output.



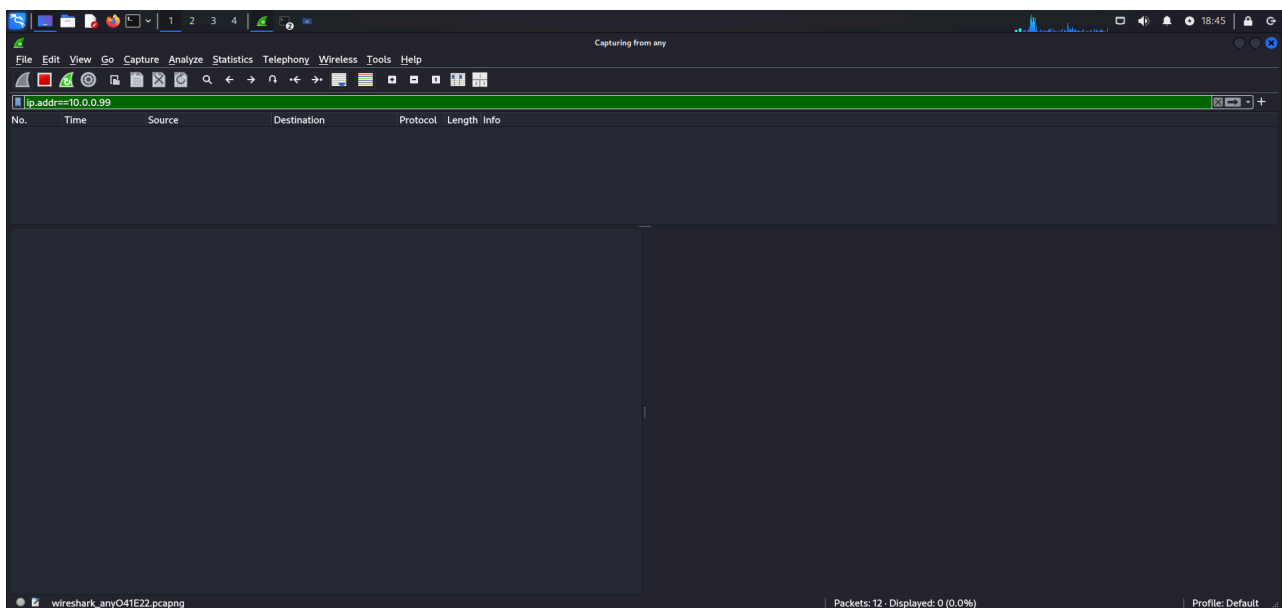
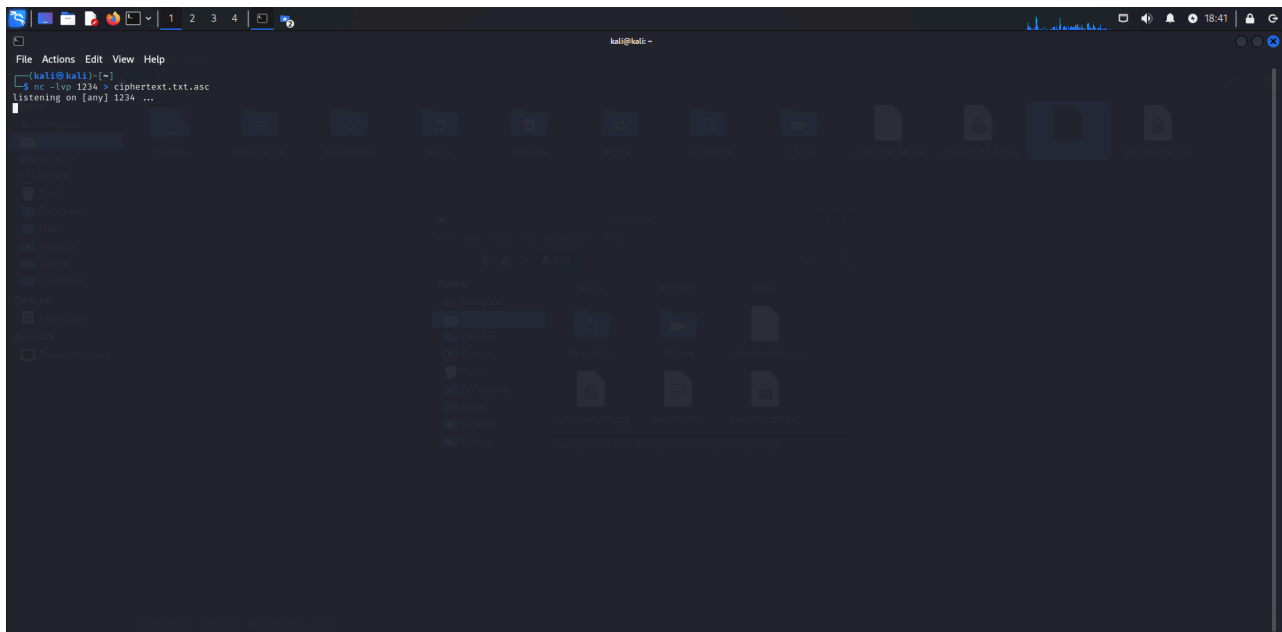
In the picture above you could see that the plain text file is symmetrically encrypted as plaintext.txt.gpg.

We rename plaintext.txt.gpg to ciphertext.txt.gpg using the (mv plaintext.txt.gpg ciphertext.txt.gpg) command. After that, we use the (cat ciphertext.txt.gpg) command to display the binary encryption of (ciphertext.txt.gpg). Next, we are generating an encryption to ASCII format using the (gpg --symmetric -a plaintext.txt) command, which generates a new file named (plaintext.txt.asc). Finally, by using the cat command on the .asc file, we reveal the ASCII-encrypted output.



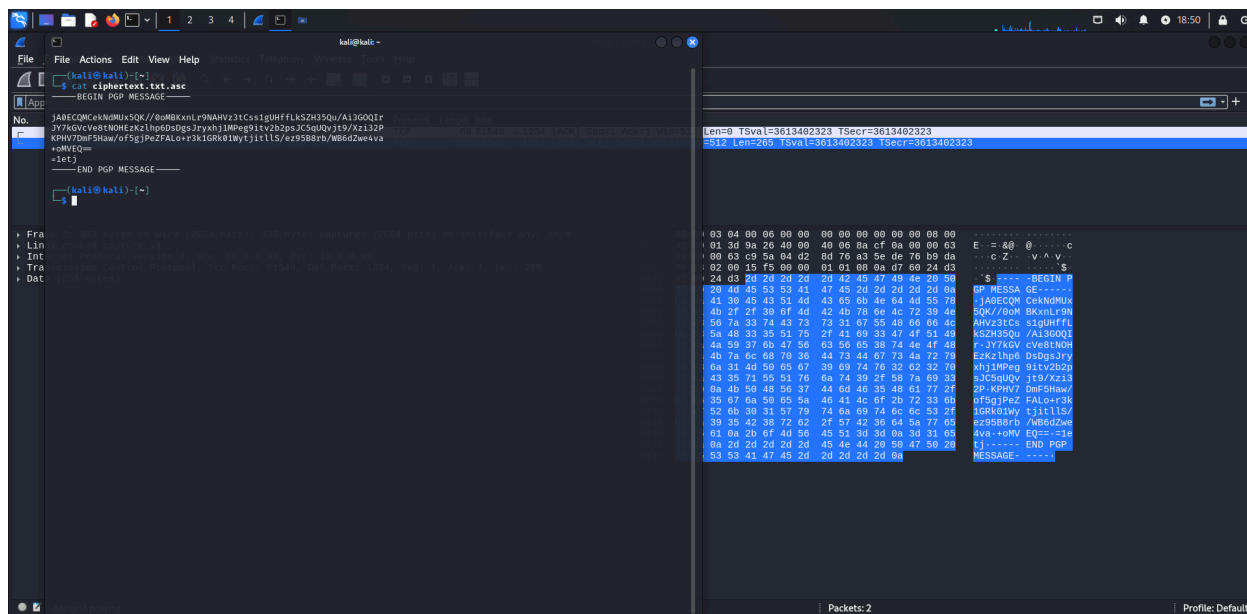
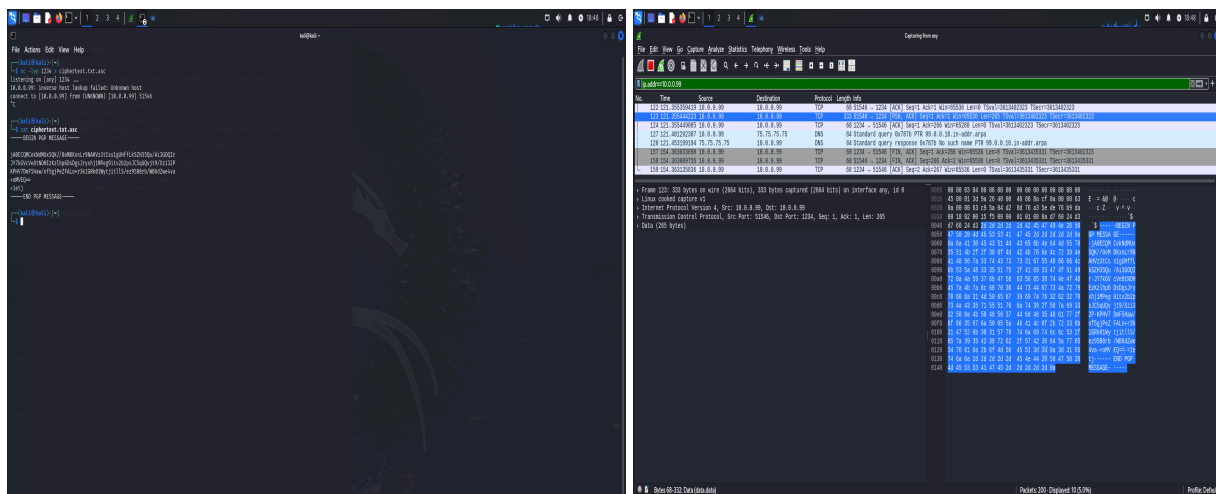
The first image on top shows the encryption done in binary encryption and also the creation of the .asc file, In the second image we could see the revealed ASCII encrypted format.

Now we are setting up a Netcat listener as `nc -lvp 1234 > ciphertext.txt.asc`. This command makes Netcat act as a server, listening on port 1234 for incoming data. When data is sent to this port using another Netcat sender from a different machine, the received data is saved into `ciphertext.txt.asc` instead of being displayed on the terminal. Next go to bash and type `ip addr` to know the ip address of the receiver. After that we are turning on Wireshark, and setting the wireshark to capture any file on the network and in the filter we are setting our receiver file ip to filter out packets being processed in Netcat communication. After that from the sender terminal send the ASCII encrypted file using the receivers ip: (`cat plaintext.txt.asc | nc 10.0.0.99 1234`).



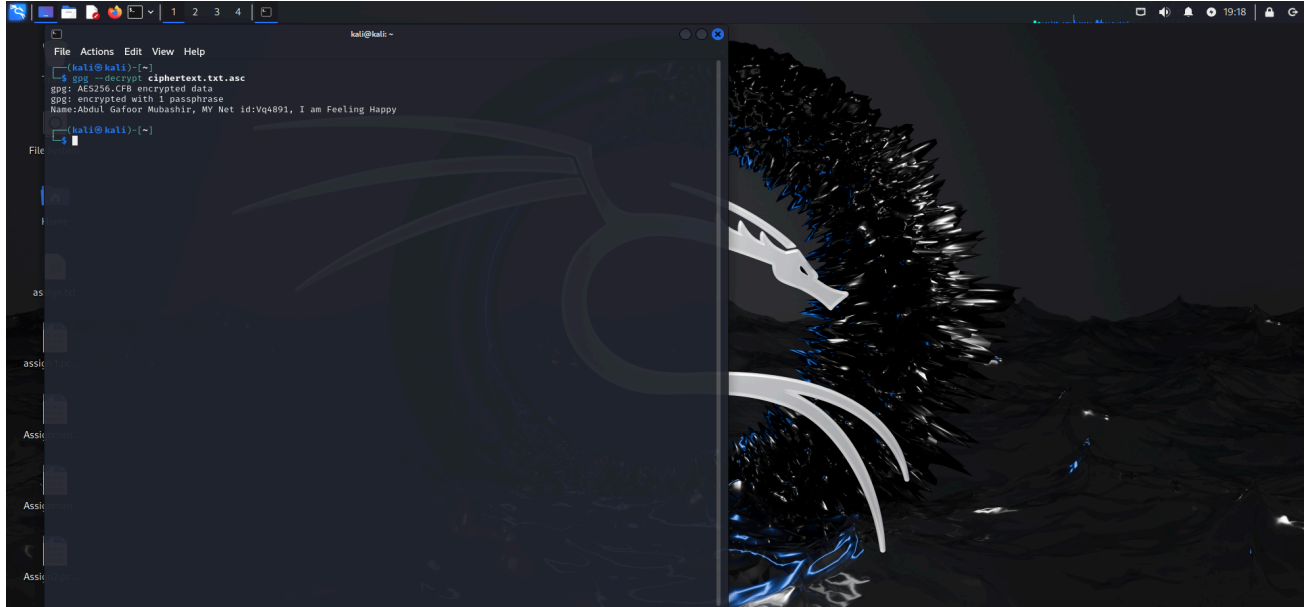
The above picture on top #1 shows the Netcat receiver terminal and #2 We have a wireshark capture terminal set to our receiver ip address.

Now, we open the received file using the cat command to verify if the ASCII-encrypted file was successfully received. Next, we analyze the captured data in Wireshark, where we can clearly observe the ASCII-encrypted PGP message within the data packet. The packet does not reveal the file's contents, but it distinctly displays the encrypted PGP message, which appears as scrambled, unreadable data from the text file we created.

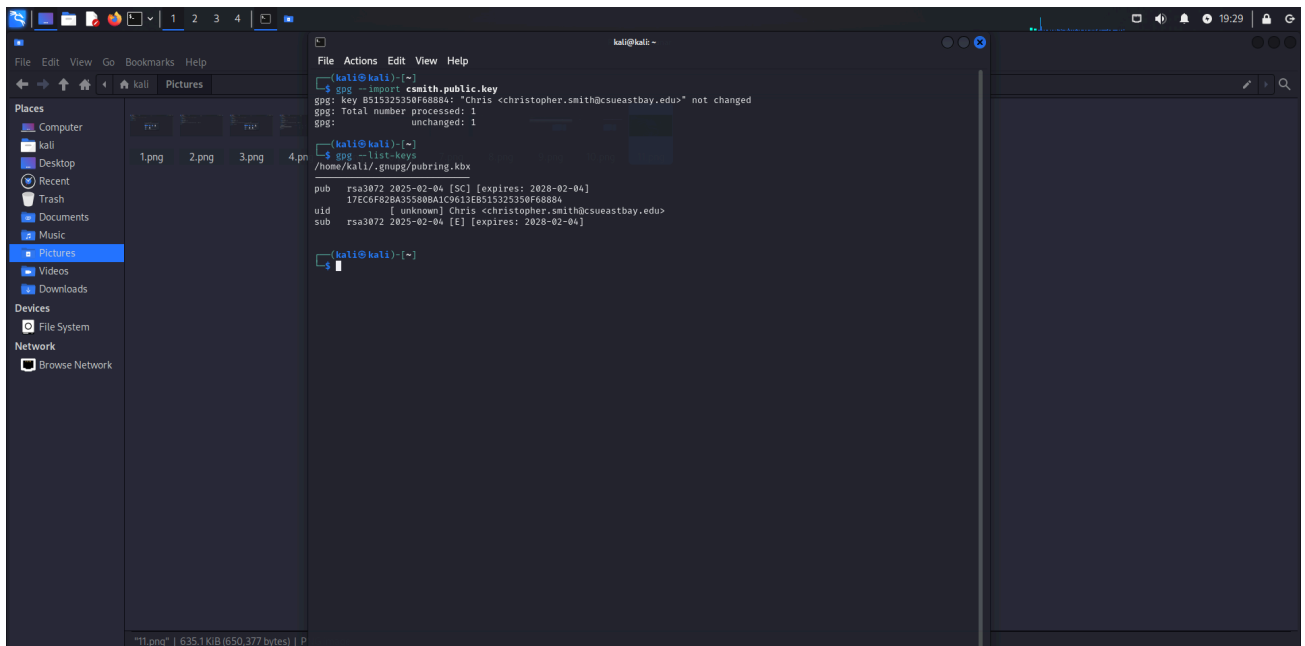


The above image is the comparison between the PGP message in the bash terminal and wireshark data packet. It is exactly the same.

Now, we have transmitted the data file and saved it as ciphertext.txt.asc from Netcat. We are going to reveal the file's content by decrypting it using the command `gpg --decrypt ciphertext.txt.asc`. Next, we are going to import the provided public key from the command `gpg --import csmith.pub.key`, this will import the public key into the GPG keychain. After import we can check for imported keys by typing `gpg --list-keys` to show the public key & user id.



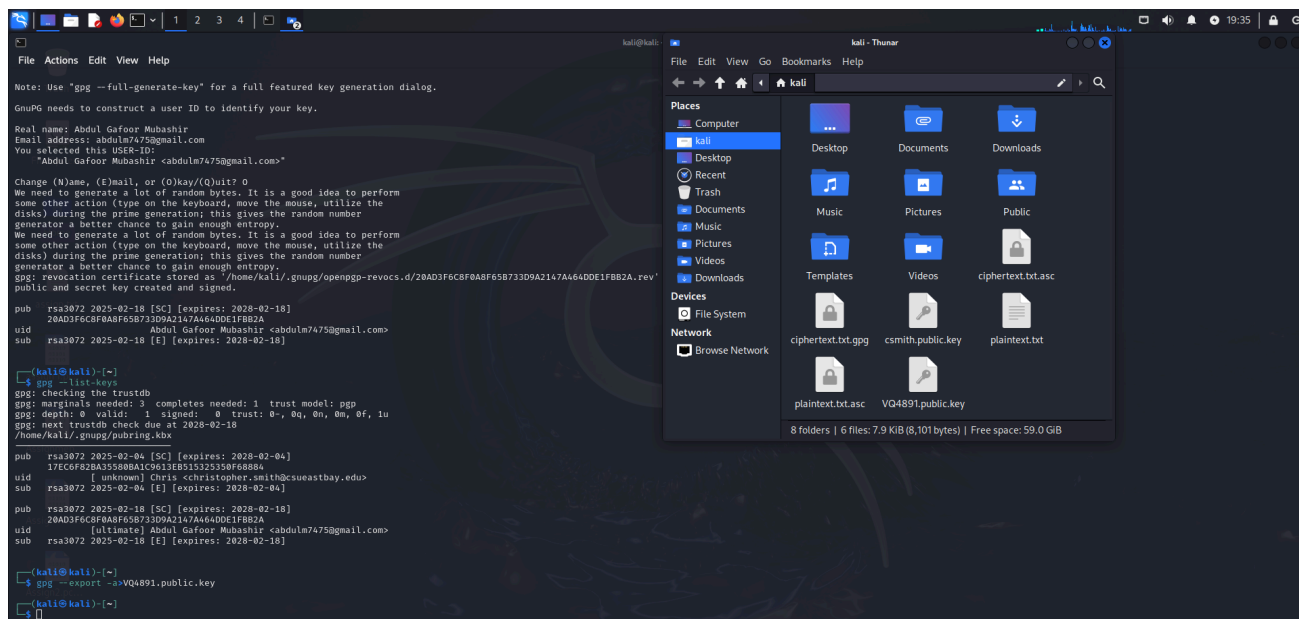
The above image shows decryption of the file.



This image shows the imported public key into the keychain.

Now, we are creating our own private and public key using gpg --gen-key command. It will ask for our Name and Email address with the desired password as a private key. Afterwards we will confirm if the key is created using the list-keys command which displays our key created with our public key and user id. We are going to export our public key as an ASCII armored file named VQ4891.public.key. And the command is: gpg --export -a > VQ4891.public.key. We can see the exported file in our folder on the second image attached below.

```
kali@kali: ~  
File Actions Edit View Help  
--(kali@kali):~  
$ gpg --gen-key  
gpg (GnuPG) 2.2.45; Copyright (C) 2024 g10 Code GmbH  
This is free software; you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
  
Note: Use "gpg --full-generate-key" for a full featured key generation dialog.  
  
GnuPG needs to construct a user ID to identify your key.  
  
Real name: Abdul Gafoor Mubashir  
Email address: abdulm7475@gmail.com  
You selected this USER-ID:  
"Abdul Gafoor Mubashir <abdulm7475@gmail.com>"  
  
Change (N)ame, (E)mail, or (O)kay/(Q)uit? 0  
We need to generate a lot of random bytes. It is a good idea to perform  
some other action (type on the keyboard, move the mouse, utilize the  
disks) during the prime generation; this gives the random number  
generator a better chance to gain enough entropy.  
We need to generate a lot of random bytes. It is a good idea to perform  
some other action (type on the keyboard, move the mouse, utilize the  
disks) during the prime generation; this gives the random number  
generator a better chance to gain enough entropy.  
gpg: revocation certificate stored as '/home/kali/.gnupg/openpgp-revocs.d/20AD3F6C8F8AF65B733D9A2147A64DDE1FB82A.rev'  
public and secret key created and signed.  
  
pub rsa3072 2025-02-18 [SC] [expires: 2028-02-18]  
20AD3F6C8F8AF65B733D9A2147A64DDE1FB82A  
uid Abdul Gafoor Mubashir <abdulm7475@gmail.com>  
sub rsa3072 2025-02-18 [E] [expires: 2028-02-18]  
  
--(kali@kali):~  
$ gpg --list-keys  
gpg: checking the trustdb  
gpg: marginals needed: 3 completes needed: 1 trust model: pgp  
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u  
gpg: next trustdb check due at 2028-02-18  
/home/kali/.gnupg/pubring.kbx  
  
pub rsa3072 2025-02-04 [SC] [expires: 2028-02-04]  
17EC6F82BA35580BA1C9613EB515325350F68864  
uid [ unknown ] Chris <christopher.smith@csueastbay.edu>  
sub rsa3072 2025-02-04 [E] [expires: 2028-02-04]  
  
pub rsa3072 2025-02-18 [SC] [expires: 2028-02-18]  
20AD3F6C8F8AF65B733D9A2147A64DDE1FB82A  
uid [ultimate] Abdul Gafoor Mubashir <abdulm7475@gmail.com>  
sub rsa3072 2025-02-18 [E] [expires: 2028-02-18]
```



```

kali@kali:~$
File Actions Edit View Help

kali@kali:~$
kali@kali:~$ gpg --list-keys
gpg: checking the trustdb
gpg: marginalis needed: 3 completes needed: 1 trust model: pgo
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0q, 0n, 0m, 0f, 1u
gpg: next trustdb check due at 2028-02-18
/home/kali/.gnupg/pubring.kbx

pub  rsa3072 2025-02-04 [SC] [expires: 2028-02-04]
     17EC6F82BA35888A1C9613EB51325358F6888A
uid          [ unknown] Chris <christopher.smith@csueastbay.edu>
sub  rsa3072 2025-02-04 [E] [expires: 2028-02-04]

pub  rsa3072 2025-02-18 [SC] [expires: 2028-02-18]
     28AD376C8F808F65873DDA2147A464D0E1F8B2A
uid          [ultimate] Abdul Gafoor Mubashir <abudum7475@gmail.com>
sub  rsa3072 2025-02-18 [E] [expires: 2028-02-18]

kali@kali:~$
kali@kali:~$ gpg --export -a VQ4891.public.key
kali@kali:~$
kali@kali:~$ gpg -a --output plaintext.txt.asc.sig --sign plaintext.txt
kali@kali:~$
kali@kali:~$ gpg --e -a -u "28AD376C8F808F65873DDA2147A464D0E1F8B2A" -r "17EC6F82BA35888A1C9613EB51325358F6888A" plaintext.txt.asc.sig
gpg: 1343348189407EF1: There is no assurance this key belongs to the named user

sub  rsa3072/1343348189407EF1 2025-02-04 Chris <christopher.smith@csueastbay.edu>
Primary key fingerprint: 17EC 6F82 BA35 5888 A1C9 613E B515 3253 58F6 888A
Subkey fingerprint: A548 088D 858F 8E52 9872 E93D 1243 3481 8940 7EF1

It is NOT certain that the key belongs to the person named
in the user ID. If you really know what you are doing,
you may answer the next question with yes.

Use this key anyway? (y/N) y

kali@kali:~$
kali@kali:~$ sudo apt-get install steghide
[sudo] password for kali:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
steghide is already the newest version (0.5.1-15).
0 upgraded, 0 newly installed, 0 to remove and 1231 not upgraded.

kali@kali:~$

```

The screenshot shows a Kali Linux terminal window on the left and a Thunar file manager window on the right. The terminal window displays the following commands and output:

```
kali@kali:~$ sudo apt-get install steghide
[sudo] password for kali:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
steghide is already the newest version (0.5.1-15).
0 upgraded, 0 newly installed, 0 to remove and 1231 not upgraded.

kali@kali:~$ wget https://i.pinimg.com/originals/be/96/ce/be96ce958f2e678cb57b38c26ceca42.jpg
2025-02-17 23:34:33-- https://i.pinimg.com/originals/be/96/ce/be96ce958f2e678cb57b38c26ceca42.jpg
Resolving i.pinimg.com (i.pinimg.com)... 208.44e22f:80, 151.101.200.84
Connecting to i.pinimg.com (i.pinimg.com)[208.44e22f:80]: 801:443...
failed: Connection timed out.
Connecting to i.pinimg.com (i.pinimg.com)[151.101.200.84]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 177609 (173K) [image/jpeg]
Saving to: 'be96ce958f2e678cb57b38c26ceca42.jpg'

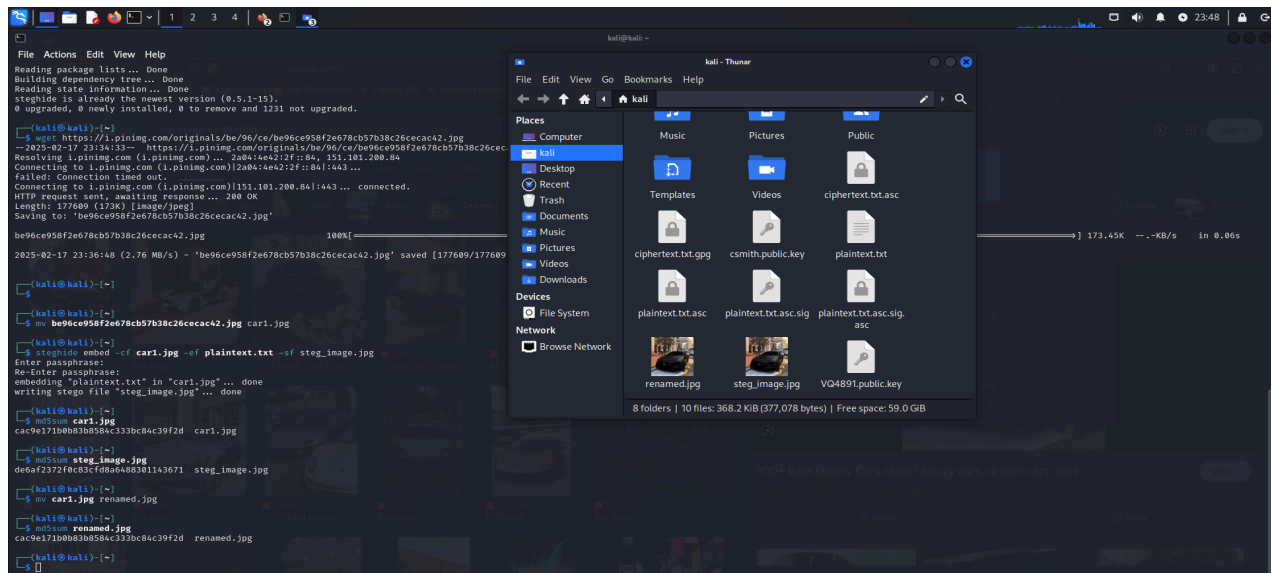
be96ce958f2e678cb57b38c26ceca42.jpg
100%[=====] 173.45K --.-KB/s in 0.06s

2025-02-17 23:36:48 (2.76 MB/s) - 'be96ce958f2e678cb57b38c26ceca42.jpg' saved [177609/177609]
```

The Thunar file manager window shows the 'Downloads' folder selected, displaying the downloaded key file 'be96ce958f2e678cb57b38c26ceca42.jpg'.

The above image shows that steghide is installed and image is downloaded and renamed to car1.jpg.

We will use the Steghide command (`steghide embed -cf car1.jpg -ef plaintext.txt -sf steg_image.jpg`) to embed the plaintext.txt file into the downloaded image. This process will generate a new image file called `steg_image.jpg`. Although both the original and the embedded image may appear identical when viewed, we can observe that the size of the embedded image is larger. To verify the integrity of the images, we can compare their hash values using the command `md5sum`. Upon doing so, we'll find that the hashes for both images are different, indicating that the embedding process has altered the file. After that we will rename the original file and compare hash value, we will observe that hash values of both are same because hash value is based only on the content not the name of the file. After that we are extracting the data from the embedded file using (`steghide extract -sf steg_image.jpg`), this will extract the data from the embedded file and save it in the folder. .



```
kali@kali:~$ curl https://i.pinimg.com/originals/be/96/be96ce958f2e678cb57b38c26ceca42.jpg -o car1.jpg
2025-02-17 23:36:48 (2.76 MB/s) = "be96ce958f2e678cb57b38c26ceca42.jpg" saved [177609/177609]

kali@kali:~$ md5sum car1.jpg
cac9e171db0b3b8584c333bc84c39f2d car1.jpg

kali@kali:~$ steghide embed -cf car1.jpg -ef plaintext.txt -sf steg_image.jpg
Enter passphrase:
Re-Enter passphrase:
Embedding "plaintext.txt" in "car1.jpg"... done
Writing stego file "steg_image.jpg"... done

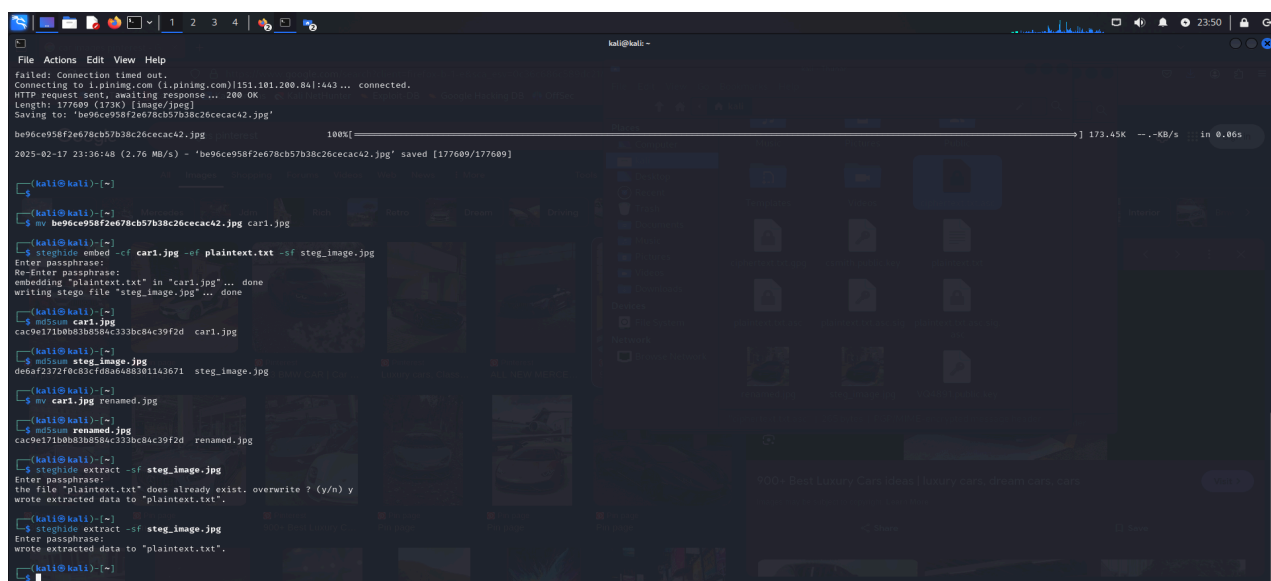
kali@kali:~$ md5sum car1.jpg
cac9e171db0b3b8584c333bc84c39f2d car1.jpg

kali@kali:~$ md5sum steg_image.jpg
defaf2372f0c83cf08a6a83b1143671 steg_image.jpg

kali@kali:~$ mv car1.jpg renamed.jpg

kali@kali:~$ md5sum renamed.jpg
cac9e171db0b3b8584c333bc84c39f2d renamed.jpg

kali@kali:~$
```



```
kali@kali:~$ steghide extract -sf steg_image.jpg
Enter passphrase:
The file "plaintext.txt" does already exist. Overwrite? (y/n) y
Wrote extracted data to "plaintext.txt".

kali@kali:~$ steghide extract -sf steg_image.jpg
Enter passphrase:
Wrote extracted data to "plaintext.txt".

kali@kali:~$
```

The image shows the data is being extracted from the embedded image.

Conclusion:

Throughout the project we got a clear understanding of doing symmetric, asymmetric encryption using GPG(GNU Privacy Guard); Steganography using steghide and hashing using the md5sum in Kali Linux. GPG provides both symmetric and asymmetric encryption, allowing users to securely encrypt and decrypt files. It also offers features such as digital signatures, which ensure authentication and non-repudiation, and the encryption ensures access control and data confidentiality, thus since we sign the file before encrypting it, the signature will not match if the data is altered which also ensures data integrity. On the other hand, Steghide is a steganography tool that hides messages within image or audio files. It asks for a passphrase before the embedded data is extracted, which offers data confidentiality by encrypting the embedded data in the file, making it harder to detect. Steghide doesn't provide any authentication and lacks access control, and also does not provide any data integrity. Moreover, Steghide doesn't verify the sender's identity or guarantee that the message came from a particular source which clearly shows it lacks non-repudiation. md5sum is a hash function which only provides hash values of the files, any modification in the file can result in different hash values. This tool only ensures the data integrity of the file. This doesn't provide other x800services except data integrity. Together, these tools can complement each other to provide a comprehensive approach to data security, with GPG handling encryption and authentication, Steghide offering covert data embedding, and md5sum ensuring data integrity checks. However, each tool has its limitations and should be used in appropriate contexts depending on the security requirements.